데이터과학 파이썬

심송용(한림대학교 데이터과학스쿨)

http://jupiter.hallym.ac.kr

자료구조

딕셔너리(dictionary): 이름이 의미하는 것과 같이 사전과 같은 형태로 키(key)와 value의 짝으로 구성된 자료구조.

- 세트와 같이 { }에 자료를 설정하여 생성할 수 있으나, key:value의 짝으로 구성된 값이라야 하며 key는 중복이 허용되지 않음.
- 특정한 위치의 값은 key값에 의해 호출
- keys 메소드, values 및 items 메소드로 각각 키, 값 및 키와 값의 짝을 볼 수 있음.

```
# dict1.py
c_code = {'Korea': 82, 'US':1, 'China':86, 'Germay':81}
print(c_code['Korea'])
print(c_code.keys())
print(c_code.values())
print(c_code.items())
```

```
82
```

자료구조

```
dict_keys(['Korea', 'US', 'China', 'Germay'])
dict_values([82, 1, 86, 81])
dict_items([('Korea', 82), ('US', 1), ('China', 86), ('Germay', 81)])

for k, v in c_code.items():
    print("key: ", k, "value: ", v)
key:  Korea value:  82
key:  US value:  1
key:  China value:  86
key:  Germay value:  81
```

keys와 values는 리스트, items는 세트를 원소로 갖는 리스트

```
if 'Korea' in c_code.keys():
    print('Korea exists.')
else:
    print('Add country code for Korea.')
Korea exists.
```

자료구조

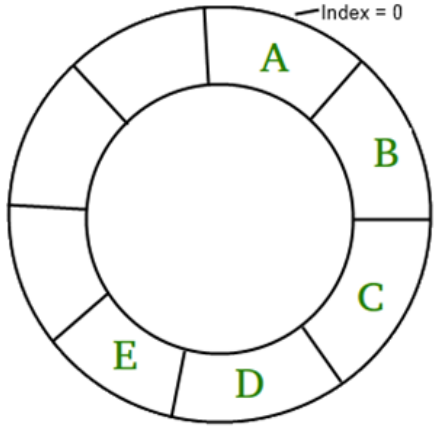
collections 모듈은 앞에서 본 리스트, 튜플, 세트, 딕셔너리 등의 기본 기능을 확장하 모듈로 자료의 처리에 유용한 기능을 제공한다. 모듈을 사용하기 위해서는

import collections as cln
과 같이 collections를 전체를 inport하고 필요한 속성을 가져 오거나 교재에서와 같이
from collections import deque
와 같이 사용한다. 전자의 경우 필요한 속성은 cln.deque와 같이 점(.)을 사용하여 호출할 수 있고 후자의 경우 deque를 직접 import 하였으므로 deque를 바로 사용할 수 있다.

collections 모듈에서 제공하는 기능중 deque, OrderedDict, defaultdict, Counter, namedtuple에 대해서 간단히 살펴보기로 하자.

자료구조

deque: deque는 리스트와 유사한 형태로 자료를 처리하며, 스택, 큐 및 circular array 기능을 제공한다. circular array는 일반적인 형태의 배열과 달리 마지막 원소 다음의 원소가 첫 번째 원소인 배열을 말한다.



Circular representation of an array

A	B	C	D	E			

자료구조

```
# collections1.py
import collections as clnl

ldeque = cln.deque()
for i in range(5):
  ldeque.append(i)
print(ldeque)

b = ldeque.pop()
print(b, ldeque)
deque([0, 1, 2, 3, 4])
4 deque([0, 1, 2, 3])
```

deque 는 pop 메소드가 항상 마지막 원소만 추출한다.(인덱스 사용불가). 하지만 appendleft나 popleft 메소드를 사용하여 처음 원소를 뽑을 수 있다.

자료구조

```
ldeque = cln.deque()
for i in range(5):
  ldeque.appendleft(i)
print(ldeque)
b = ldeque.popleft(); print(b, ldeque)
deque([4, 3, 2, 1, 0])
4 deque([3, 2, 1, 0])
```

circular array(또는 cyclic array) 기능이 있어서 회전이 가능.

```
ldeque.appendleft(4); print(ldeque) # 원래대로 [4,3,2,1,0]
ldeque.rotate(2); print(ldeque)
ldeque.rotate(-2); print(ldeque)

deque([4, 3, 2, 1, 0])
deque([1, 0, 4, 3, 2])
deque([4, 3, 2, 1, 0])
```

자료구조

OrderedDict: 기본 딕셔너리에서 입력한 순서를 기억하는 딕셔너리이다.

```
import collections as cln # collections2.py

d1 = {}
d1['x'] = 100; d1['l'] = 500; d1['y'] = 200; d1['z'] = 300
for k, v in d1.items(): print(k, v)
```

```
x 100
l 500
y 200
z 300
```

의 결과는 컴퓨터에 따라 출력의 순서와 입력의 순서가 같지 않을 수 있다. OrderedDict를 사용하면

```
d2 = cln.OrderedDict()
d2['x'] = 100; d2['y'] = 200; d2['z'] = 300; d2['l'] = 500
for k, v in d2.items(): print(k, v)
```

```
x 100
```

자료구조

y 200

z 300

l 500

입력한 순서를 기억한다.(이 경우 입력순서는 값의 크기 순)

딕셔너리의 정렬

sorted 함수는 파이썬에 내장된 함수로 주어진 iterable을 key값 기준으로 정렬하는 함수이다. 이 함수는

```
sorted(iterable, key=None, reverse=False)
```

로 사용하며 크기순 정렬에 사용할 함수를 설정하며 reverse는 True이면 오름차순 False이면 내림차순으로 정렬한다.

따라서

```
def sort_by_key(mydict):
    return mydict[0]
```

라는 함수가 정의되어 있고 d가 딕셔너리라면

자료구조

sorted(d.items(), key=sort_by_key)

명령에 의해 0번째 d.items의 0번째 차원(key)에 의해서 정렬된다. 만일 d의 value에 의해서 정렬하고 싶다면

```
def sort_by_value(mydict):
    return mydict[1]
```

를 사용하면 된다. (두번째 필드가 value)

```
def sort_by_key(mydict):
    return mydict[0]
def sort_by_value(mydict):
    return mydict[1]

d = {} # 일반 딕셔너리 또는 d=dict() 로 선언
d['x'] = 100; d['l'] = 500; d['y'] = 200; d['z'] = 300
print("key에 의한 정렬")
for k, v in cln.OrderedDict(sorted(d.items(), key=sort_by_key)).items():
    print(k,v)
```

자료구조

```
print("value에 의한 정렬(내림차순)")
for k, v in cln.OrderedDict(sorted(d.items(), key=sort_by_value, reverse=True)).items():
    print(k,v)
```

key에 의한 정렬

l 500

x 100

y 200

z 300

value에 의한 정렬(내림차순)

l 500

z 300

y 200

x 100